Amusement Park (Solution)

We consider the graph whose vertices are the attractions in JOIOI park, and edges are the paths. Since this graph is connected, we can take a spanning tree. Hence it is enough to solve the task if the graph is a tree.

Let us consider the following strategy to tell the integer X:

- We put an integer between 0 and 59, inclusive, to each vertex. The way to do it should be the same for JOI-kun and IOI-chan.
- To each vertex, JOI-kun writes the value of a bit of X. The position of the bit is the integer of that vertex.
- For each integer between 0 and 59, inclusive, IOI-chan visits a vertex corresponding to it. Then she can recover the value of X.

By the following method, we can put an integer between 0 and 59, inclusive, to each vertex so that the following condition is satisfied:

> for each vertex, there is a subtree containing it such that the subtree also contains exactly one vertex corresponding to each integer between 0 and 59, inclusive.

The method is as follows:

1. To any vertex r, we can assign a subtree T_0 of size 60 containing it. For example, this can be achieved by the depth-first search from the vertex r.
2. To each of the vertices of T_0, we put an integer between 0 and 59, inclusive. Different vertices of T_0 should have different integers. Moreover, to each of these vertices, we assign the subtree T_0.
3. Assume that u, v are adjacent vertices such that a subtree T is assigned to u, and v is not contained in T. If we did not yet put an integer to v, we can put an integer and assign a subtree to v by the following way:
   - Take a leaf w of T different from u.
   - Let T' be the subtree obtained from T by removing w and appending v.
   - Put an integer of w to v. Assign the subtree T' to v.
4. Repeat the procedure 3 until we put an integer to every vertex.

By the above method, we also assign a required subtree for each vertex.

IOI-chan performs the depth-first search for this subtree. She can visit all vertices of the subtree by at most $2(|T| - 1) = 118$ moves.

Therefore, we can read all of the 0–59h bits of X. We can recover X successfully, and get full score.

Bulldozer (Solution)

In the following, we consider rock as gold of negative value.

Subtask 1

First, we consider the case where all spots lie on a line (Subtask 1). We sort the spots according to their x-coordinates. Let $W_1, W_2, \ldots, W_N$ be the value of gold obtained at each spot, in order.

We put $S_0 = 0$ and $S_i = W_1 + W_2 + \cdots + W_i$. Then the maximum profit is $\max\{ S_j - S_i \mid 0 \leqq i < j \leqq N \}$.

Subtask 2

We consider the general case. If we choose the slope a of a line, by taking the projections of all spots to a line of slope $-1/a$, the problem is reduced to the case where all spots lie on a line. (The case of a line parallel to the y-axis is the same as the case of a line of sufficiently large slope.)

Let us consider how the order of spots are changed when a varies. If we change the value of a from $-\infty$ to $+\infty$, the order is changed only when $aX_i - Y_i = aX_j - Y_j$ is satisfied for some $i \neq j$.

Therefore, since there are at most $N(N-1)/2$ candidates of a, we can solve Subtask 2.

Subtasks 3–5

We consider Subtask 3. Note that, if we change the value of a from $-\infty$ to $+\infty$, when the order is changed, only two adjacent spots are interchanged at one moment. Hence this subtask is solved if we can manage the maximum, the minimum, and $\max\{ S_j - S_i \mid j > i \}$ of the intervals of the sequence $\{ S_i \}_{0 \leq i \leq N}$. We can implement it using a **segment tree**.

We can solve Subtask 4 if we refer the segment tree for $\max\{ S_j - S_i \mid 0 \leq i < j \leq N \}$ only when all changes of the same slopes are done.

If more than three points lie on a line, we can similarly solve the task if we devise the order to change the spots. Then we get full score.

Golf (Solution)

First, by coordinate compression, we may assume that the coordinates are at most $2N$.

Moreover, the answer will not be changed if we further assume that the golf ball passes only the lines extending one of the sides of an obstacle, and the lines parallel to one of the coordinates passing through the start point or the end point. The reason is, if a given route of the golf ball does not satisfy this condition, we will get a valid route if we replace a move in the route by the nearest line by parallel translation because there is no obstacle between them.

Therefore, we extend line segments from each side of each obstacle, the start point, and the end point until they meet other obstacles, and search on them by the **breadth-first search**.

To get full score, we efficiently need to

- enumerate the line segments connected with the current line segment, and
- manage line segments so that we visit each line segment at most once.

For example, this can be implemented by the following way.

We divide each line segment so that we put it on a **segment tree**. Then, in total, the line segments are divided into $O(N \log N)$ parts.

We manage divided line segments by constructing a segment tree for each type of intervals covered by parts of the line segments.

At each interior point of the segment tree, we put the number of line segments in the corresponding interval to which we did not yet visit. At each leaf node, if there is a line segment to which we did not yet visit, we put information about it.

For each line segment parallel to the x-axis, there are at most $O(\log N)$ types of intervals of the y-coordinates of line segments meeting it. While visiting line segments, we do the following: search the nodes of the segment tree corresponding to the interval as long as there is a line segment to which we did not yet visit, and, when we reach a leaf node, we consider the line segment in that node as a candidate of the next state, and remove it from the segment tree.

The total time complexity for search is $O(N \log^2 N)$. We can get full score by this method.